TYPES OF HASH FUNCTION

Hash functions converts key into their respective index positions in hash tables.

Following are the different types of hash functions.

1. Division Method
2. Multiplication Method
3. Extraction Method
4. Mid-Square Method
5. Folding Method
6. Universal Method

1. Division Method:

This is the most simple and easiest method to generate a hash value for any data item k which you want top store in the hash table.

The hash function divides the value k by M and then uses the remainder obtained.

h(K) = k mod M

Here, k is the key value, and

M is the size of the hash table.

It is best suited that M is a prime number as that can make sure that keys are more uniformly distributed.

1. Division Method Example

k = 12345
M = 95
h(12345) = 12345 mod 95 = 90

k = 1276
M = 11
h(1276) = 1276 mod 11= 0

Advantage-

This method is good for any value of M.
It is fast as it requires only single division operation.

Disadvantage

This method leads to poor performance since consecutive keys map to consecutive indices in the hash table.

2.Multiplication method

This method involves the following steps:

1. Choose a constant value A such that $0 < A < 1$.
2. Multiply the key value with A.
3. Extract the fractional part of kA.
4. Multiply the result of the above step by the size of the hash table i.e. M.
5. The resulting hash value is obtained by taking the floor of the result obtained in step 4.

2. Multiplication method

Formula:

h(K) = floor (M (kA mod 1))

Here,

M is the size of the hash table.

k is the key value.

A is a constant value.

Where "k A mod 1" means the **fractional part of k A**

2. Multiplication method :

Example:

k = 12345

A = 0.357840

M = 100

h(12345) = floor[100 (12345*0.357840 mod 1)]

= floor[100 (4417.5348 mod 1)]

= floor[100 (0.5348)]

= floor[53.48]

= 53

2. Multiplication method :

Advantage-

1.It can work with any value in between 0 to 1.

Disadvantage-

1.The multiplication method is generally suitable when table size is power of two. If it is so then multiplication hashing very fast.

3. Extraction Method

In this method we have to extract part of a key and remaining digits are ignored.

Extracted part simply works as hash key.

How many digits are extracted it is dependent on size of the hash table.

E,g. If table size=100(index from 0 to 99) then we can extract two digit from the key.

Example:-

Key =12345

r= number of extracted digit =2 so we first two digit are extracted and h(key)=12

3. Extraction Method

Advantages-

1.Simple to calculate.

Disadvantages-

1. Key may not be evenly distributed throughout the table.

2.The mid square method

The mid square method is a very good hashing method. It involves two steps to compute the hash value-

Square the value of the key k i.e. k^2

Extract the middle r digits as the hash value.

Formula:

h(K) = h(k x k)

Here,

k is the key value.

Example:

Suppose the hash table has 100 memory locations.

So r = 2 because two digits are required to map the key to the memory location.

k = 60

k x k = 60 x 60

= 3600

h(60) = 60

The hash value obtained is 60

4.The mid square method

Advantages-

1.All digits of key value contributes to result generation.

Disadvantages-

1.The size of key is one of the limitation of this method.

2.There will be more changes of collision.

5. Digit Folding Method

This method involves two steps:

Divide the key-value k into a number of parts i.e. k1, k2, k3,….,kn, where each part has the same number of digits except for the last part that can have lesser digits than the other parts.

Add the individual parts. The hash value is obtained by ignoring the last carry if any.

Formula:

k = k1, k2, k3, k4, ….., kn

s = k1+ k2 + k3 + k4+..kn

h(K)= s % M

(where M is table of Size)

Here, s is obtained by adding the parts of the key k

5. Digit Folding Method

Example:

k = 12345

k1 = 12, k2 = 34, k3 = 5

s = k1 + k2 + k3

= 12 + 34 + 5

= 51

h(K) = 51

h(K) = 51 % 10 =1

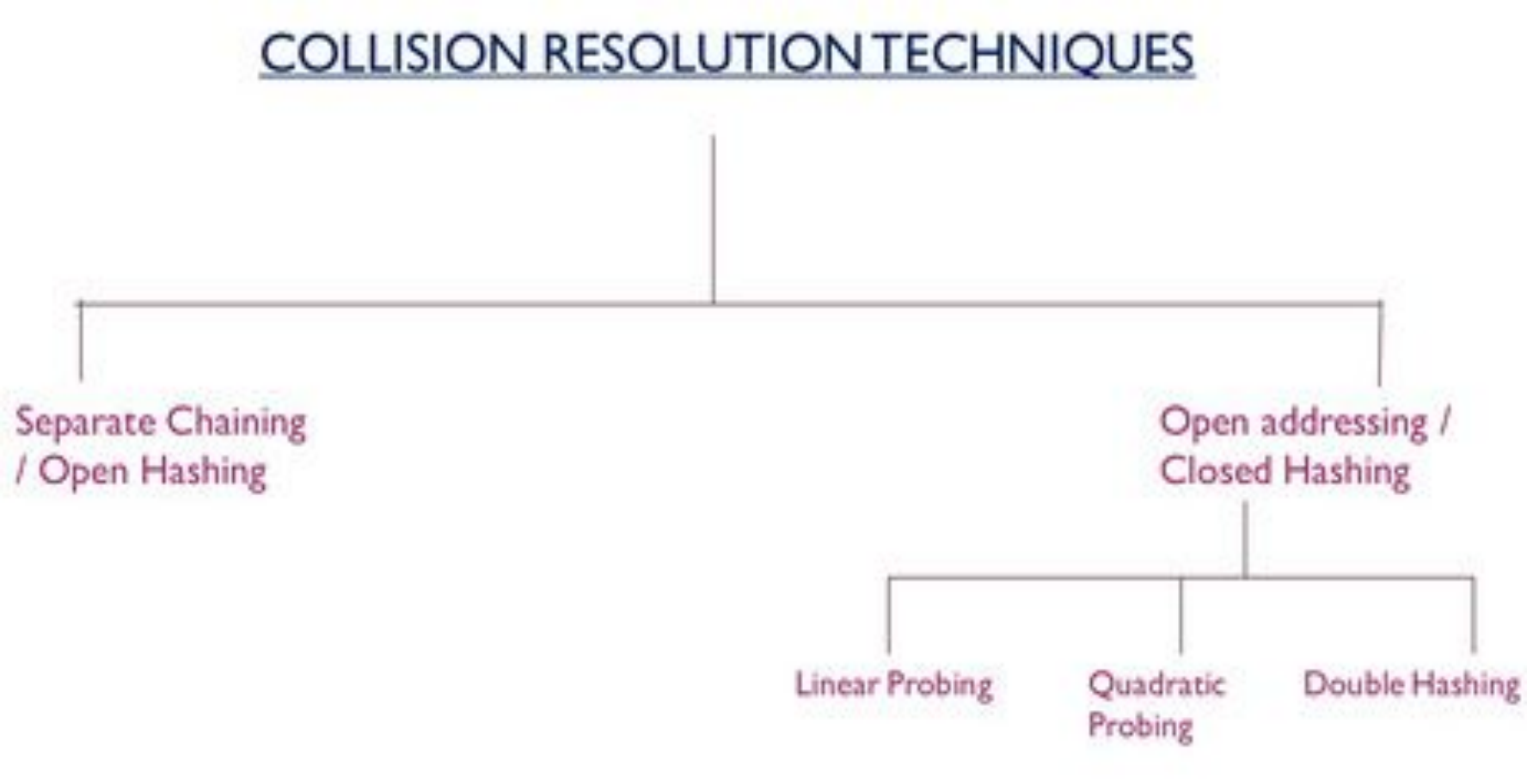
6.Universal Method

If refers to selecting a hash function at random from the family of hash functions with a certain mathematical property.

Advantages-

1.It ensures minimum number of collision.

Collision resolution strategies:-



Open Hashing / Separate Chaining

In Open Hashing, one of the methods used to resolve the collision is known as a Sseparate chaining method.

In Open Hashing , we maintain a linked list at every hash index for colliding keys.

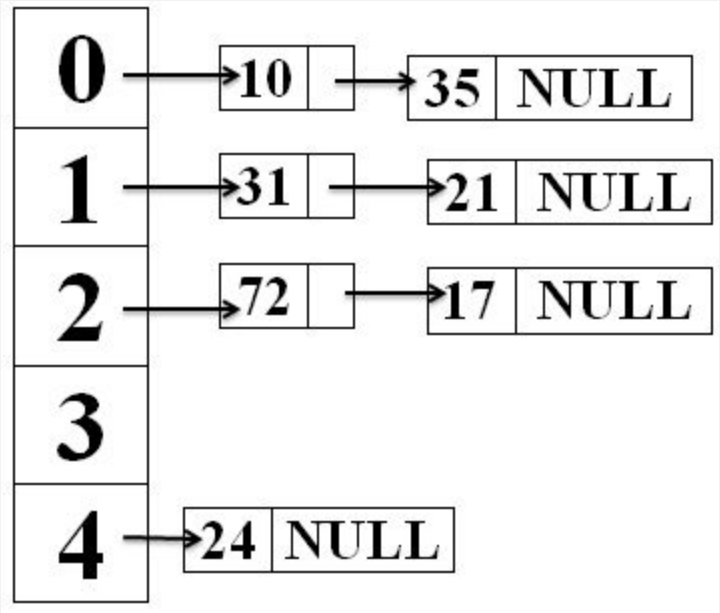
Suppose we wish to store a set of number {10,31,21.72,24,35,17 } into hash table of size 5.

Assume that we have hash function H such that **H(key)=key % 5.**

So we will get respective indices for given key values as follows after applying hash function on keys.

Open Hashing / Separate Chaining

H(10)=10 % 5=0
H(31)=31 % 5=1
H(21)=21 % 5=1
H(72)=72 % 5=2
H(24)=24 % 5=4
H(35)=35 % 5=0
H(17)=17 % 5=2



Clearly 10& 35 , 31 & 21 as well as 72 &17 will have same hash value means they are mapped to same index.

In this case we simply append the colliding values to a list pointed by their hash value as shown in diagram.

Open Hashing / Separate Chaining

Advantages-

1. Open hashing method is simple to implement & understand.
2. The hash table never ends, so we can add new elements always.

Disadvantages-

1. The cache performance is poor as the key are stored using singly linked list.

2. The storage space is wasted as some part of hash table is never used.

3. Extra storage is used for storing links of the chain.

Closed hashing /Open addressing

These collision resolution techniques required a hash table with fixed & known size.

When we want to insert any key if index calculated for that key is already taken (i.e. collision occurs) then alternate cells are tried until empty position is found.

This technique requires the size of hash table to be supposedly greater than number of keys to be inserted / stored.

There are three different techniques to find the alternate position for colliding keys

1. Linear Probing 2. Quadratic Probing 3. Double Hashing

Linear Probing

The idea behind Linear Probing simple we a fixed sized hash table & whenever collision occurs we traverse linearly / Sequential in cyclic manner to find next empty slot.

It mean when we want to insert key & if a key is mapped (hashed) to an index which is already occupied then check for next available position to place them. i.e.

H(key) = h, h(k)+1,h(k)+2,h(k)+3,…….,h(k)+I

Example-

let us consider a simples= hash function as key % table Size which means,

Hash(key)=key % S where s□ size of the table.

Let us consider s=7 & index value in table are form 0 to 6.

Linear Probing

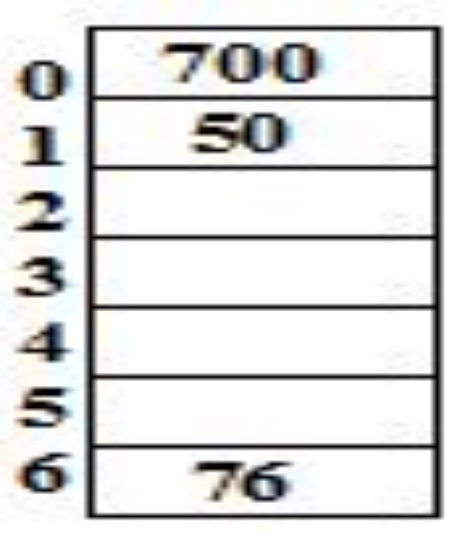
Now if we want to insert following value in table
(50,700,76,85,92,73,101)
Index / hash key is calculated for every key as follows
50 % 7 = 1
700 %7 = 0
76 % 7 = 6
85 % 7 =1
92% 7 =1
73 %7=3
101 % 7 =3

Linear Probing

1) When we try to insert key ‘50’ it is inserted at index position 1.

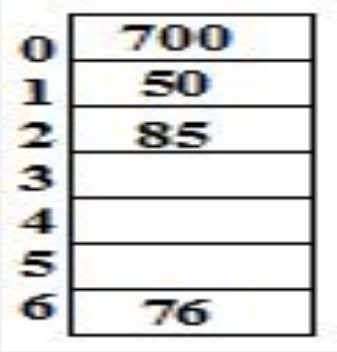
2) Similarly key ‘700’ is inserted at index position 0.

3) Similarly key ‘76’ is inserted at index position 6 as shown in the hash table step 3.

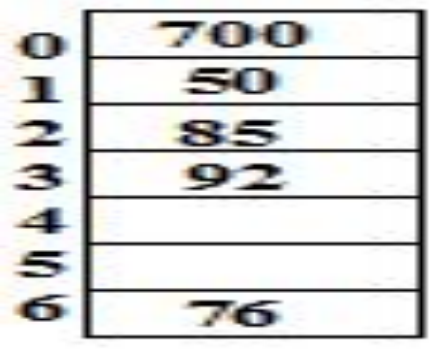


Linear Probing

4) When we try to insert ‘85’ since position calculated for ‘85’ is 1 collision occurs as ‘50’ is already present at position 1 so we store ‘85’ at next empty position i.e. at index position 2.



5) When we try to insert ‘92’ since position calculated for ‘92’ is 1 again collision occurs as ‘50’ is already present at position 1 so we store ‘92’ at next empty position i.e. at index position 3



Linear Probing

6) When we try to insert ‘73’ since position calculated for ‘92’ is 3 collision occurs as ‘92’ is already present at position 3 so we store ‘73’ at next empty position i.e. at index position 4.

0	700
1	50
2	85
3	92
4	73
5	
6	76

7) Finally while inserting ‘101’ since index position ‘3’ is already occupied to avoid collision we need to insert to insert at next empty position i.e.5.

0	700
1	50
2	85
3	92
4	73
5	101
6	76

Linear Probing

Key	Location /Position	Probe
50	50 % 7 = 1	1
700	700 %7 = 0	1
76	76 % 7 = 6	1
85	85 % 7 =1	2
92	92% 7 =1	3
73	73 %7=3	2
101	101 % 7 =3	3
Total Probes		13

Quadratic Probing

It is one of the technique in open addressing (closed hashing). In this method when any key collides then insert that key new location we look up for $(i*i)^{th}$ slot i.e.(i^2) slot in every i^{th} iteration.

We start looking from original hash location. If only the location is occupied then we check for other slots.

Let hash(x) be the slot index computed using hash function. If slot hash(x)% S is full the we will try for [hash(x)+1*1]%S if this slot also full then try at [hash(x)+2*2]%S if this is also full then we will try for [hash(x)+3*3]%S & we can continue like this until size of the hash table.

$H(k)=h, H(k)+1^2, H(k)+2^2, H(k)+3^2 \ldots\ldots\ldots H(k)+S^2$

Quadratic Probing

Example:-

Let us consider a simple hash function as key % table size which means

h(key)= key % S, let S=10 & we want to insert values (9,19,29,39,49,71)

1.h(09)=09%10=9 available insert at 9 index position in the hash table.

2.h(19)=19%10=9 collision so h(19)=((h(09)+1^2)%10 = ((9+1)%10) = 0 available so insert at '19' at 0th index position in the hash table.

3.h(29)=29%10=9 collision so h(29)=((h(29)+1^2)%10 = ((9+1)%10) = 0 again collision so h(29)=((h(29)+2^2)%10 = ((9+4)%10) = 3 available so insert at '29' at 3rd index position in the hash table.

Quadratic Probing

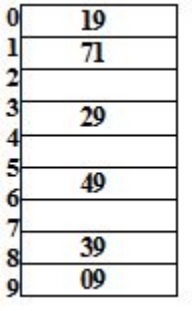
4. h(39)=39%10=9 collision so

$h(39)=((h(39)+1^2)\%10 = ((9+1)\%10) = 0$ again collision so

$h(39)=((h(39)+2^2)\%10 = ((9+4)\%10) = 3$ so again collision occurs

$h(39)=((h(39)+3^2)\%10 = ((9+9)\%10) = 8$

available so insert at '39' at 8th index position in the hash table.



Quadratic Probing

5. h(49)=49%10=9 collision so

h(49)=((h(39)+1^2)%10 = ((9+1)%10) = 0 again collision so

h(49)=((h(9)+2^2)%10 = ((9+4)%10) = 3 so again collision occurs

h(49)=((h(49)+3^2)%10 = ((9+9)%10) = 8 again collision occurs so

h(49)=((h(49)+4^2)%10 = ((9+16)%10) = 5 available so insert at '49' at 5th index position in the hash table.

6.h(71)=h(71)%10=1 available so insert '71' at 1st index position in the hash table.

0	19
1	71
2	
3	29
4	
5	49
6	
7	
8	39
9	09

Double Hashing

Double Hashing is technique in which a second hash function is applied to key when collision occurs.

By applying the second hash function we will get the number of position from the point of collision to insert.

There are two important rules to be followed for the second function.

1. It must never evaluate to ZERO.
2. Must make sure that all the cell can be probed.

H_1(key) = key % Table Size

H_2(key) =(M-(key % M))

Where M is the a prime number smaller than the size of the table.

$Hash(x)=(Hash_1(x)+ i * Hash_2(x))\% S$

Double Hashing

Example Consider the following elements to be placed in the hash table of size 10. {37,90,45,22,49,17,55} .

Initially , insert the elements using the formula H_1(key) = key % Table Size

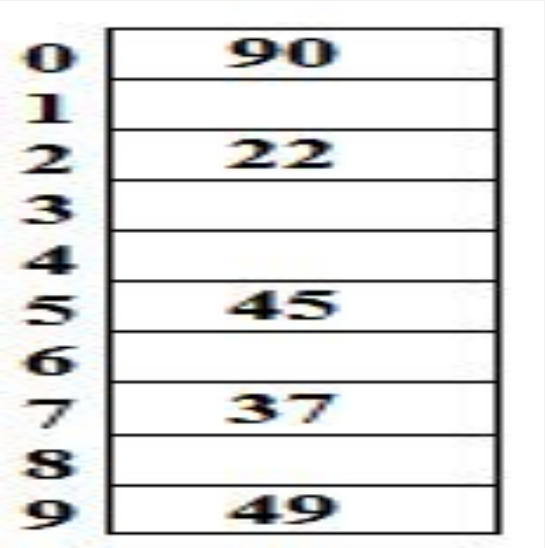
$H_1(37) = 37 \% 10 = 7$

$H_1(90) = 90 \% 10 = 0$

$H_1(45) = 45 \% 10 = 5$

$H_1(22) = 22 \% 10 = 2$

$H_1(49) = 49 \% 10 = 9$



Now, $H_1(17) = 17 \% 10 = 7$ collision so

$H_2(17) = (7-(17 \% 7)) = 7-3 = 4$

Hash(x)=(Hash$_1$(x)+ i * Hash$_2$(x))% S where i: The attempt number, usually 1,2,3,…so on

Hash(x)=(7+ 1 *4)% 10=(11%10)=1

therefore ‘17’ is placed at 1st index position in the hash table.

Double Hashing

Now, $H_1(55) = 55 \% 10 = 5$ collision so

$H_2(55) = (7-(55 \% 7)) = 7-6 = 1$

$Hash(x) = (Hash_1(x) + i * Hash_2(x)) \% S$

$Hash(x) = (5 + 1 * 1) \% 10 = 6 \% 10 = 6$

Therefore '55' is placed at 6th index position in the hash table.

0	90
1	17
2	22
3	
4	
5	45
6	55
7	37
8	
9	49

Advantages of Double hashing

1. The advantage of Double hashing is that it is one of the best form of probing, producing a uniform distribution of records throughout a hash table.

2. This technique does not yield any clusters.

3. It is one of effective method for resolving collisions

Advantages of Collision Resolution Techniques

1) Simple to implement.
2) Hash table never fills up, we can always add more elements to the chain.
3) Less sensitive to the hash function or load factors.
4) It is mostly used when it is unknown how many and how frequently keys may be inserted or deleted. .

Disadvantages of Collision Resolution Techniques

1) Cache performance of chaining is not good as keys are stored using a linked list. Open addressing provides better cache performance as everything is stored in the same table.

2) Wastage of Space (Some Parts of hash table are never used)
3) If the chain becomes long, then search time can become O(n) in the worst case.

4) Uses extra space for links

Differentiae between Separate Chaining and Open Addressing

Separate Chaining	Open Addressing
It is also called as Open Hashing	It is also called as Closed Hashing
Keys are stored inside the hash table as well as outside the hash table.	All the keys are stored only inside the hash table.
The number of keys to be stored in the hash table can even exceed the size of the hash table.	The number of keys to be stored in the hash table can never exceed the size of the hash table.
Deletion is easier.	Deletion is difficult.
Extra space is required for the pointers to store the keys outside the hash table.	No extra space is required.
Cache performance is poor. This is because of linked lists which store the keys outside the hash table.	Cache performance is better. This is because here no linked lists are used.
Some buckets of the hash table are never used which leads to wastage of space.	Buckets may be used even if no key maps to those particular buckets.

THANK YOU!!!